

T11.2 - Arquitectura Lambda - Introducción

Ecuación

→ Un sistema de datos responde mirando las preguntas del pasado.

Cualquier base de datos relacional almacena datos que ya ocurrieron. Uno le hace consultas sobre eso: mirando los datos del pasado, se consigue la respuesta.

→ Entonces el sistema de datos más general responde preguntas mirando el conjunto de datos completo.

Para generalizar más, una base de datos no se debe limitar a un subconjunto (datos del último mes o de una tabla específica), sino mirando todos los datos que existen.

Por lo tanto, un sistema de datos se puede definir con la *función*:

```
query = function(all data)
```

Arquitectura lambda

→ Es una arquitectura pensada para *resolver los problemas de Fully Incremental y Event Sourcing* para *casos de uso no triviales*.

Fully Incremental guarda la verdad más reciente y Event Sourcing guarda todos los eventos inmutables desde siempre. Cada uno tenía distintos problemas tanto de costo como complejidad.

Un caso de uso no trivial es un escenario donde se necesita procesar volúmenes a nivel del Big Data y también tener data freshness (brecha entre "lo que está pasando ahora" y lo que ve el sistema) a la vez.

→ Combina *estrategias de procesamiento batch y real-time*.

Batch: procesa todos los datos de un saque.

Real-time: procesa los datos a medida que van llegando.

→ Permite *aplicar técnicas de Big Data a problemas que requieren data freshness*. En esa dirección.

Esta arquitectura está *organizada en capas*:

Capa	Tarea
Speed Layer	<i>Construye real-time views</i> . → Procesa los datos nuevos que van llegando en tiempo real. → Construye las real-time views. → Cubre la brecha temporal entre el último batch y el momento actual. A diferencia del batch layer que recalcula todo desde cero, <i>esta capa es</i>

Capa	Tarea
	<i>incremental</i> . Cada vez que el batch layer termina un nuevo ciclo, la speed layer puede descartar lo que ya fue incorporado al batch.
Serving Layer	<i>Sirve batch views</i> . → Toma las batch views. → Las sirve para que sean consultables. Es una capa de lectura que expone las vistas precomputadas.
Batch Layer	<i>Construye batch views</i> . → Almacena el master dataset con sus datos crudos e inmutables. → Ejecuta funciones sobre ese dataset para generar las batch views. Es el cómputo pesado y lento.

La función `query = function(all data)` podría ser muy cara. Por eso, tiene sentido *pre-computar la función query*.

Ese pre-cálculo se llama **batch view** → los resultados de *ven* de un cálculo previo.

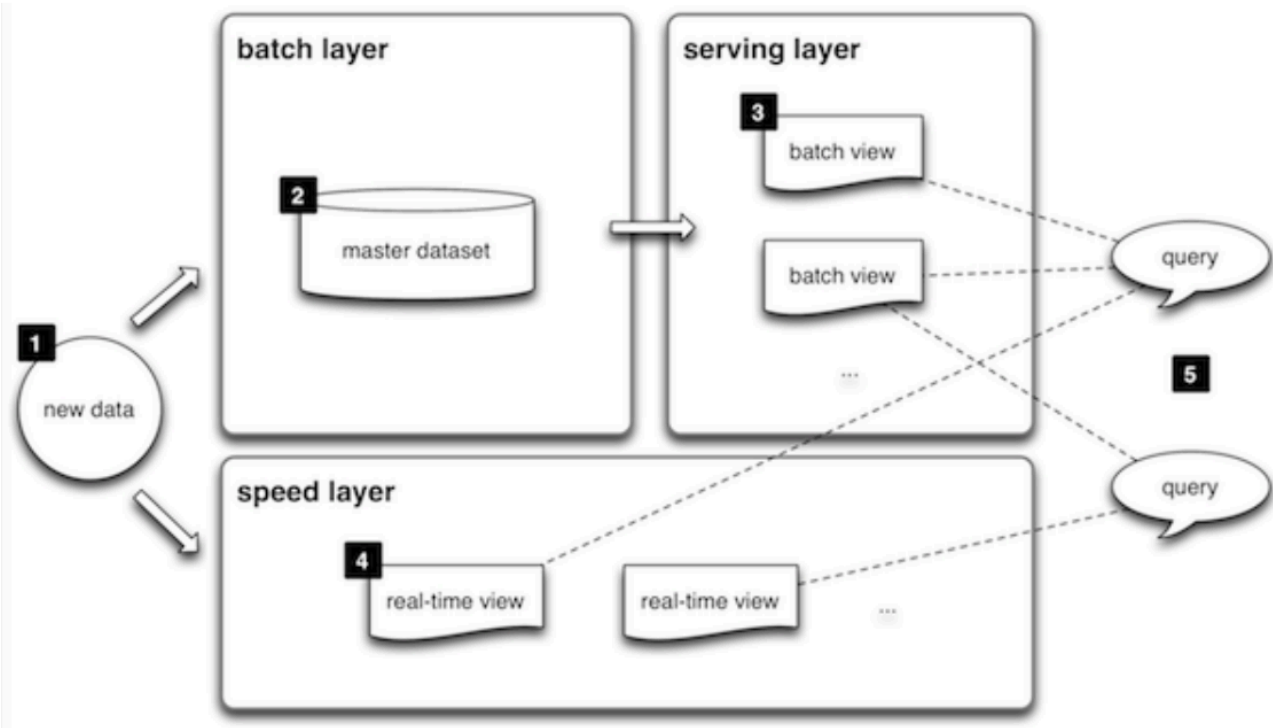
```
batch view = function(all data)
```

Esta es una operación de alta latencia porque se ejecuta una función sobre todos los datos. El batch view resultante ni siquiera tendrá todos los datos.

Una **realtime view** *incorpora todos esos datos que quedaron afuera* del batch view.

```
realtime view = function(realtime view, new data)
```

De ahora en más no se usa más la función `query = function(all data)` para explicar qué es la arquitectura lambda.



1. Llega un dato nuevo. Se envía a dos lugares en simultáneo: al *batch layer* y al *speed layer*.
2. El batch layer lo agrega al master dataset. Queda almacenado ahí. Cada tanto, el batch layer *procesa todo ese master dataset* y genera batch views.
 Recordar que el master dataset es el almacén completo de todos los datos crudos que el sistema recibió desde siempre, sin modificar. Solamente se agregan datos sobre él, no se borran ni sea actualizan. Si algo sale mal, siempre se puede acudir a esta fuente de verdad.
3. Las batch views se pasan a la serving layer. *Esta capa las indexa* y las deja *listas para ser consultadas*. El índice es necesario para encontrar rápido esa view.
4. En simultáneo, la speed layer toma ese mismo dato nuevo y *actualiza las real-time views* de forma inmediata → incremental.
5. Cuando llega una query, *se consulta la serving layer y también la speed layer*, o sea las batch views y las real-time views respectivamente. Se combinan ambos resultados para dar la respuesta completa.
 Se combinan porque cada view por sí sola es incompleta, no cubre todo el panorama completo.
 La batch view tiene el análisis profundo que se generó en algún momento, pero le faltan los datos más recientes.
 La real-time view tiene los datos frescos pero solo cubre lo que pasó desde el último batch. Si se juntan las dos, se cubre todo el rango temporal.